

DSCI 523 Quiz 1

Import, names, pipe

- CSV: `readr::read_csv("file.csv")` -> tibble (URL allowed). Delimited: `readr::read_delim(path, delim = "\t", col_names = FALSE)`; set delimiter; no header otherwise consumes row 1 as names. Metadata/preview: `read_csv(path, skip = 2, n_max = 100)` (`n_max` = data rows). Excel: `readxl::read_excel(path, sheet = "Sheet1")`; .xlsx is not CSV; download URL first. Write: `readr::write_csv(df, "out.csv")` (names, no row names).
- Check delimiter/header/metadata/column count/types. Backticks for `Mother tongue`; `rename(df, new = old)`; many names: `janitor::clean_names()`.
- `<-` assigns; `==` compares. `median(x = 1:10)`: named argument; `median(x <- 1:10)`: also assigns. `|>` passes left side as first argument; `dplyr` takes/returns a table and uses bare column names.
- Debug a pipe by letting it print first; `assign(result <-) only` after its intermediate/result shape is right. `%>%` is the older pipe; current code uses `|>`.

dplyr and tidy shape

```
flights |> filter(carrier %in% c("AC", "WS"), delay > 0) |>
  mutate(delay_hours = delay / 60) |> select(carrier,
delay_hours) |>
  arrange(desc(delay_hours))
```

- `select(df, a, first:last)`: same rows. `filter(df, c1, c2)`: all conditions TRUE; comma = AND; NA rows drop. Set: `x %in% c("A", "B")`; elementwise OR/AND: `|`, `&`, `!not |`, `&&` (only element 1).
- `mutate(df, new = old * 2, old = round(old))`: add/overwrite; later expressions may use earlier ones. `arrange(df, x, desc(y))`; `slice(df, 1)` remains table; sort then slice for extreme row. `pull(df, col)` -> vector.
- **Tidy**: one observation/row, variable/column, value/cell. Define the observation first.

```
wide |> pivot_longer(`1999`:`2000`, names_to = "year",
values_to = "cases")
wide |> pivot_longer(~country, names_to = "year",
values_to = "cases")
long |> pivot_wider(names_from = type, values_from =
count)
```

- Column names are values -> `pivot_longer(cols, names_to, values_to)`; `cols` uses `select()` syntax (`~id, everything()`). Values in a column become names -> `pivot_wider(names_from, values_from)`; remaining columns identify the observation.
- Shape check: `country | 1999 | 2000` becomes `country | year | cases` with `pivot_longer()`; `country | type | count` becomes `country | A | B | ...` with `pivot_wider()`. A remaining identifier column is part of the observation; an accidental extra identifier prevents intended rows from combining.
- `pivot_longer(~country, ...)` keeps `country` as the identifier; `pivot_longer(everything(), ...)` would also turn `country` values into the names column. `pivot_wider()` uses **all** columns not named in `names_from/values_from` as identifiers—select/keep only the columns that define one desired output row.

R values, types, extraction

- Scalar = length-1 vector. Atomic vectors are homogeneous: logical (1L), double (1), character; `c(1, "a")` -> character. Factor has finite levels (whose order affects graphs/statistics); list may mix/nest; matrix is homogeneous 2-D; data frame is equal-length columns. Tibbles are data-frame subclasses; `as_tibble(df)` converts explicitly.
- Check `typeof(x)` (underlying type), `class(x)` (important for factor/data frame/tibble), `str(x)` (compact type/length/content), `is.integer(x)`, `as.double(x)` (may lose information). Index starts at 1.
- `v[2:4]/v[-1]` -> vector. `lst[1]` -> list; `lst[[1]]` -> element; nest `lst[[6]][[3]]`. `df[rows, cols]/df[1]` -> table; `df[["mass"]]/df$mass` -> column. `df[, 1]`: base data frame simplifies to vector, tibble retains tibble.
- `y <- x`, then modify `y`: `x` unchanged (copy-on-modify). Arithmetic/comparisons and `&` `|` `!` are elementwise; short inputs recycle (non-multiple lengths warn). `&&`/`||` inspect element 1. NA is missing, not a value: numeric/logical operations can yield NA; in `filter()`, conditions must be TRUE, so NA rows drop.
- Matrix indexing is `[row, column]`. A data frame is a special list of equal-length column vectors: `df[1:5, c("conc", "uptake")]` remains a table; `df[df$ cyl == 6, ]` is base-R logical row filtering. In data-operation questions, use `tidyverse filter()` unless base R is explicitly requested.

Dates, strings, factors

```
d <- ymd("2017-01-31"); dt <- ymd_hms("2017-01-31
20:11:59")
mdy_hm("01/31/2017 08:01"); make_date(year, month, day)
make_datetime(year, month, day, hour, minute, second)
year(dt); month(dt); mday(dt); yday(dt); wday(dt, label =
TRUE, abbr = FALSE)
term <- interval(start, end)
filter(holidays, holiday_date %within% term)
```

- `ymd/mdy/dmy` -> Date; time helpers -> date-time (tz adds time zone). `today()` -> Date. Parse character dates before comparison; strings are not reliable dates. `%within%` returns one logical per date, so it works directly inside `filter()`.

- Parser name follows input order: use `dmy()` for day-month-year, `mdy()` for month-day-year; they accept character or unquoted numeric dates. Parse serialized text with a parser; build separate components with `make_date()`/`make_datetime()`.
- `str_*` is vectorized. `str_detect(x, "^A")` -> logical (`==` exact only); `str_subset()` -> vector; `str_split()` -> list (variable pieces); `str_split_fixed(..., n=2)` -> matrix. `str_length()` is per-string count; `str_sub(x, 1, 3)` (assign through `str_sub(...)` <- "AAA"). `str_c(a,b, sep="-")` elementwise; `str_c(x, collapse="-")` one string; `str_replace()` replaces first match—use `mutate(col = str_replace(col, ...))` to replace a table column.
- Regex: `^` start, `$` end, `.` any; regex backslash in an R string: `"\\"`. `separate()` splits a table column; `unite()` combines.
- Clean character text before `factor()`, rather than treating a factor like free text. Check with `class()/str()`, `levels(f)`, `nlevels(f)`; filter leaves unused levels -> `fct_drop(f)`. `fct_infreq()`, `fct_reorder(f, x, .fun=median)`, `fct_relevel(f, "Asia", "Africa")`, `fct_rev()` set level/graph/statistical order.
- `str_length(c("cat", "horse"))` -> `c(3, 5)`, not the vector length. `str_c(c("a", "b"), c("1", "2"), sep="-")` -> `c("a-1", "b-2")`; adding `collapse="-"` instead reduces all elements to one string. `filter(df, str_detect(col, "tan$"))` keeps rows whose col ends in tan.

Two tables

`left_join(x, y, by = c("countries" = "country"))`; `x` is left/main; same-name keys join by default. Duplicate keys create all combinations: two `x` rows and three `y` rows with the same key yield six matched rows—check key uniqueness and output rows.

- `left_join`: all `x` + `y` columns; unmatched `y` = NA. `inner_join`: matched rows only + both columns. `full_join`: all rows/columns; unmatched side = NA.
- `semi_join`: matched `x` only; only `x` columns; duplicate `y` does not copy `x`. `anti_join`: unmatched `x`; only `x` columns. First three are mutating; last two filtering: use `semi/anti` when the right table decides membership but should add no columns.
- `bind_rows()` stacks corresponding variables/types. `bind_cols()` matches row position only—prefer a keyed join unless alignment is guaranteed.
- Join debugging: identify the key and expected output rows before joining; inspect repeated keys in **both** tables, then count output rows. Same-name key uses the default; different names require `by = c("x_key" = "y_key")`. A right-only key never appears in `left_join()`, but does appear in `full_join()`.

Base control flow and final check

```
if (x > t) { a } else if (x < t) { b } else { c }
for (item in x) print(item ^ 2)
for (i in seq_along(x)) print(x[i] ^ 2)
```

- `if` needs one logical value (not a whole column/vector); `{}` for multiple statements. `for (item in x)` iterates values; use `seq_along(x)` plus `x[i]` only when position matters. Never `1:length(x)` for `character()`, it is `1:0` but `seq_along()` is empty.
- Answer shape first (table/vector/Date/factor/value); one table -> verb; shape -> observation then pivot; two tables -> retained rows + right columns then join; convert date/string/factor types first; check names, commas, quotes, brackets, pipe.